

Compilador JavaSubset $\rightarrow$ x86-64

con Verificación Formal mediante Lógica de Hoare y Z3

Documentación Técnica del Proyecto

Diseño de Compiladores — Fases 1 a 5

Autor: nickhernd

Repositorio: <https://github.com/nickhernd/pl>

Lenguaje: C++11 (Flex + Bison)

Herramientas: GCC, Z3 SMT Solver

Fecha: 28 de junio de 2026

Resumen

Este documento describe el diseño e implementación de un compilador completo para un subconjunto de Java con destino nativo x86-64. El compilador abarca cinco milestones: análisis léxico-sintáctico con Flex y Bison, análisis semántico con tablas de símbolos y verificación de tipos, generación de código intermedio en formato de cuádruplas de tres direcciones, backend x86-64 con generación de ensamblador AT&T y asignación de registros mediante el algoritmo de coloración de grafos de Chaitin-Briggs, y finalmente un módulo de verificación formal basado en la Lógica de Hoare y el Cálculo de Precondiciones Más Débiles (*Weakest Precondition*, WP) con exportación a formato SMT-Lib2 para el solver Z3.

El compilador es capaz de procesar programas Java anotados con especificaciones formales (`@pre`, `@post`, `@invariant`, `@variant`) y generar automáticamente condiciones de verificación cuya validez es comprobada por Z3, detectando así tanto corrección de programas como errores de división por cero y no terminación.

Índice general

1. Introducción	13
1.1. Motivación y objetivos	13
1.2. Subconjunto del lenguaje	13
1.3. Herramientas utilizadas	14
1.4. Estructura del documento	14
2. Arquitectura del compilador	15
2.1. Vista global del pipeline	15
2.2. Representaciones intermedias	16
2.3. Modos de operacion	16
2.4. Patron Visitor	17
2.5. Estructura de directorios	17
3. Frontend: Análisis léxico-sintáctico	19
3.1. Análisis léxico	19
3.1.1. Categorías de tokens	19
3.1.2. Tokens especiales para verificación	19
3.2. Análisis sintáctico	20
3.2.1. Estructura gramatical principal	20
3.2.2. Gestión de anotaciones formales	20
3.3. Árbol Sintáctico Abstracto (AST)	20
3.3.1. Jerarquía de nodos	21
3.3.2. Nodos especiales para verificación	21
4. Análisis semántico	23
4.1. Tabla de símbolos	23
4.2. Comprobaciones semánticas	23
4.2.1. Reglas de coerción de tipos	24
4.3. Ejemplo de error semántico detectado	24
4.4. Visita a nodos de especificación formal	24
5. Representación intermedia	25
5.1. Motivación	25
5.2. Formato de cuádruplas	25
5.3. Argumentos IR	25
5.4. Traducción de construcciones	26
5.4.1. Expresión aritmética	26
5.4.2. Sentencia if-else	26
5.4.3. Bucle while	26

5.4.4. Llamada a función	27
5.5. Visualización de la IR	27
6. Backend x86-64	29
6.1. Convención de llamada System V AMD64 ABI	29
6.2. Marco de pila	29
6.3. Generación de código por opcode	30
6.3.1. Operaciones aritméticas enteras	30
6.3.2. Saltos condicionales	30
6.3.3. Llamadas a función	30
6.3.4. E/S con <code>printf</code> y <code>scanf</code>	30
6.4. Ejemplo completo de programa compilado	31
7. Optimizaciones sobre la representación intermedia	33
7.1. Introducción	33
7.2. Eliminación de código muerto	33
7.3. Plegado de constantes	34
7.4. Propagación de copias	34
7.5. Impacto de las optimizaciones	34
8. Asignación de registros: Chaitin-Briggs	35
8.1. Motivación	35
8.2. Registros disponibles	35
8.3. Algoritmo	35
8.3.1. Fase 1: Análisis de vivacidad	35
8.3.2. Fase 2: Grafo de interferencias	36
8.3.3. Fase 3: Coloración del grafo	36
8.4. Uso del resultado	37
9. Verificación formal: Lógica de Hoare y cálculo WP	39
9.1. Fundamentos teóricos	39
9.1.1. Lógica de Hoare	39
9.1.2. Cálculo de Precondiciones Más Débiles	39
9.2. Reglas del cálculo WP	40
9.2.1. Regla de la asignación (VER-4)	40
9.2.2. Regla de la composición (VER-5)	40
9.2.3. Regla del condicional	41
9.2.4. Regla del retorno	41
9.2.5. Regla del bucle con invariante (VER-6)	41
9.3. Jerarquía de predicados	41
9.4. Generación de condiciones de verificación (VER-7)	42
9.5. Exportación a SMT-Lib2 (VER-8)	42
9.6. Interpretación de resultados (VER-10)	43
9.7. Ejemplo completo de verificación	43
9.7.1. Función <code>absValue</code>	43
9.7.2. Función <code>wrongMethod</code> (bug intencional)	43
9.7.3. Suma con invariante de bucle	44
9.8. Nota sobre invariantes	44

10. Suite de pruebas	45
10.1. Organización	45
10.2. Pruebas de backend	45
10.3. Pruebas semánticas	46
10.4. Pruebas de verificación formal	46
10.4.1. Salida de ejemplo: <code>demo_verified.java</code>	46
10.5. Script de ejecución	46
 A. Gramática BNF del subconjunto Java	 49
A.1. Estructura del programa	49
A.2. Declaración de métodos	49
A.3. Sentencias	49
A.4. Expresiones	50
A.5. Anotaciones de verificación	50
A.6. Precedencia y asociatividad de operadores	51
 B. Referencia de instrucciones x86-64	 53
B.1. Instrucciones de transferencia de datos	53
B.2. Instrucciones aritméticas enteras	53
B.3. Instrucciones de comparación y salto	54
B.4. Instrucciones de control de flujo y llamadas	54
B.5. Instrucciones de punto flotante SSE2	55
B.6. Directivas del ensamblador	55
B.7. Convención de registro de marcos (<code>%rbp</code>)	55

Índice de figuras

2.1.	Pipeline completo del compilador JavaSubset. La rama verde (verificación formal) se activa con la opción <code>-verify</code>	16
2.2.	Jerarquía del patrón Visitor en el compilador.	17
3.1.	Jerarquía de nodos del AST (flecha = hereda de).	21

Índice de cuadros

1.1. Características del subconjunto Java soportado	13
2.1. Flags del compilador y su efecto	17
3.1. Categorías de tokens reconocidas por el lexer	19
4.1. Tabla de coerción de tipos para operaciones binarias	24
5.1. Opcodes de la representación intermedia	25
6.1. Registros de la System V AMD64 ABI	29
6.2. Traducción de operaciones aritméticas enteras	30
7.1. Operaciones que admiten plegado de constantes	34
7.2. Reducción de cuádruplas en los programas de prueba	34
8.1. Registros asignables por el compilador	35
9.1. Clases de predicados implementadas	41
9.2. Interpretación de la respuesta de Z3	43
9.3. VCs generadas para <code>sumLoop</code> y resultados de Z3	44
10.1. Grupos de pruebas	45
10.2. Pruebas de backend (compilación y ejecución)	45
10.3. Pruebas de errores semánticos	46
10.4. Pruebas de verificación con Z3	46
A.1. Precedencia (mayor nivel = mayor prioridad)	51
B.1. Instrucciones de movimiento	53
B.2. Aritmética entera (64 bits)	53
B.3. Comparación y saltos	54
B.4. Llamadas a función y retorno	54
B.5. Instrucciones SSE2 para <code>double</code>	55
B.6. Directivas GAS usadas por el compilador	55

Listings

2.1.	Organizacion del repositorio	17
3.1.	Reglas léxicas para anotaciones formales	19
3.2.	Reglas principales de la gramática Bison	20
3.3.	Vinculación de anotaciones al MethodNode	20
3.4.	Nodo WhileNode con soporte de invariante	21
4.1.	Estructura de la tabla de símbolos	23
4.2.	Reasignación de constante — error semántico	24
4.3.	Visita a ResultNode en el análisis semántico	24
5.1.	if-else en código fuente	26
6.1.	Prólogo y epílogo de función	29
6.2.	Comparación IR_LT y salto condicional	30
6.3.	Llamada a función y recogida de valor de retorno	30
6.4.	Impresión de entero con printf	30
6.5.	Lectura de entero con scanf	31
6.6.	Programa fuente: suma hasta 5	31
6.7.	Ensamblador x86-64 generado para sumToFive	31
8.1.	Análisis de vivacidad (punto fijo)	36
8.2.	Construcción del grafo de interferencias	36
8.3.	Coloración simplificada de Chaitin-Briggs	36
9.1.	wpSeq con continuaciones para retornos anticipados	40
9.2.	Sustitución en BinaryPred	42
9.3.	Formato SMT-Lib2 para una VC	42
9.4.	Método verificado: valor absoluto	43
9.5.	Método con bug intencional	43
9.6.	Suma 1..5 con invariante completo	44
A.1.	Producción raíz	49
A.2.	Métodos y parámetros	49
A.3.	Sentencias del subconjunto	49
A.4.	Gramática de expresiones (precedencia ascendente)	50

Capítulo 1

Introducción

1.1. Motivación y objetivos

El diseño e implementación de un compilador desde cero constituye uno de los proyectos más completos en el ámbito de la Ingeniería del Software, pues pone en juego simultáneamente teoría de lenguajes formales, estructuras de datos avanzadas, generación de código máquina y, en este caso, lógica matemática aplicada a la verificación de programas.

El proyecto **JavaSubset Compiler** tiene como objetivo construir un compilador completo para un subconjunto estáticamente tipado de Java con destino a código nativo x86-64. El proceso se organiza en cinco fases o *milestones*:

Fase 1. Frontend: análisis léxico y sintáctico, construcción del AST.

Fase 2. Análisis semántico: resolución de nombres, comprobación de tipos.

Fase 3. Representación intermedia: traducción a código de tres direcciones.

Fase 4. Backend x86-64: generación de ensamblador, asignación de registros.

Fase 5. Verificación formal: Lógica de Hoare, cálculo WP, integración con Z3.

1.2. Subconjunto del lenguaje

El lenguaje fuente es un subconjunto tipado de Java con las siguientes características:

Cuadro 1.1: Características del subconjunto Java soportado

Categoría	Construcciones soportadas
Tipos primitivos	<code>int</code> , <code>double</code> , <code>boolean</code>
Declaraciones	Variables locales, constantes <code>final</code>
Operadores	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , comparación, lógicos
Control de flujo	<code>if/else</code> , <code>while</code>
Métodos	Estáticos sin parámetros, <code>return</code>
E/S estándar	<code>System.out.print/println</code> , <code>Scanner</code>
Anotaciones formales	<code>@pre</code> , <code>@post</code> , <code>@invariant</code> , <code>@variant</code>

1.3. Herramientas utilizadas

Flex 2.6 Generador de analizadores léxicos a partir de expresiones regulares. Produce la función `yylex()` en C.

GNU Bison 3.x Generador de analizadores sintácticos LALR(1). Produce el autómata de análisis y las acciones semánticas en C.

GCC / G++ 11 Compilador del código C/C++ generado. Se usa también para ensamblar y enlazar el código x86-64 producido.

Z3 4.x Solver SMT de Microsoft Research. Recibe fórmulas en formato SMT-Lib2 y determina su satisfacibilidad (SAT/UNSAT).

1.4. Estructura del documento

El documento sigue la estructura del pipeline del compilador: el capítulo 2 presenta la vista global del sistema; los capítulos 3–6 describen cada fase de compilación; el capítulo 7 detalla las optimizaciones sobre la RI; el capítulo 8 explica la asignación de registros; el capítulo 9 desarrolla la verificación formal; y el capítulo 10 recoge los resultados experimentales.

Capítulo 2

Arquitectura del compilador

2.1. Vista global del pipeline

El compilador sigue la arquitectura clásica en fases consecutivas. Cada fase transforma una representación del programa en otra de mayor nivel de abstracción hacia la máquina destino, o bien extrae información (tabla de símbolos, tipos) que se usa en fases posteriores.

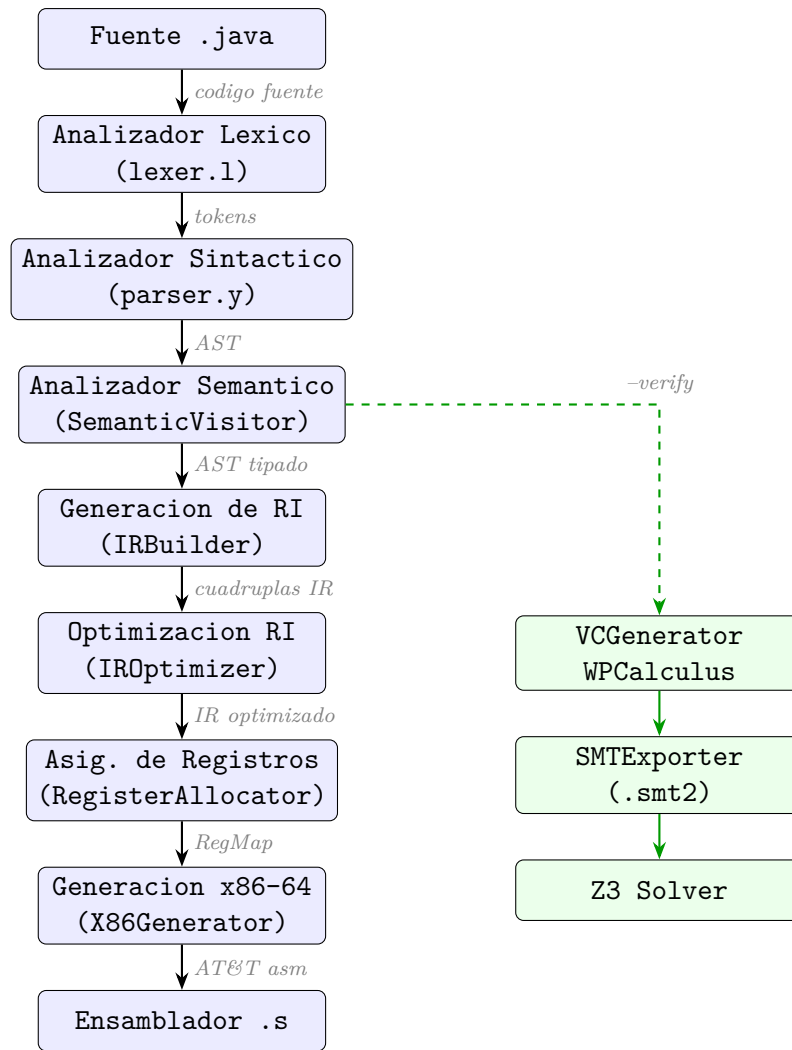


Figura 2.1: Pipeline completo del compilador JavaSubset. La rama verde (verificación formal) se activa con la opción `-verify`.

2.2. Representaciones intermedias

A lo largo del pipeline el programa adopta cuatro representaciones distintas:

1. **Código fuente:** texto Java con posibles anotaciones formales.
2. **AST** (*Abstract Syntax Tree*): árbol de nodos tipados, jerarquía de clases C++ visitables mediante el patrón *Visitor*.
3. **Cuadraplas IR:** secuencia plana de instrucciones de la forma (op, arg_1, arg_2, res) , independiente de la arquitectura.
4. **Ensamblador x86-64:** instrucciones AT&T con registros reales o con posiciones en la pila.

2.3. Modos de operacion

El compilador admite tres modos de operacion seleccionables mediante flags:

Cuadro 2.1: Flags del compilador y su efecto

Flag	Salida	Descripcion
(ninguno)	prog.s	Compilacion estandar a ensamblador
-ir	stderr	Imprime cuadruplas IR y forma lineal
-reg-alloc	stderr	Imprime mapa de asignacion de registros
-verify	prog.smt2	Verificacion formal con Z3

2.4. Patron Visitor

Todas las fases que procesan el AST utilizan el patron de diseno **Visitor** (GoF). La interfaz abstracta **Visitor** declara un metodo `visit()` para cada tipo de nodo del AST; cada fase implementa esta interfaz sin modificar las clases del arbol.

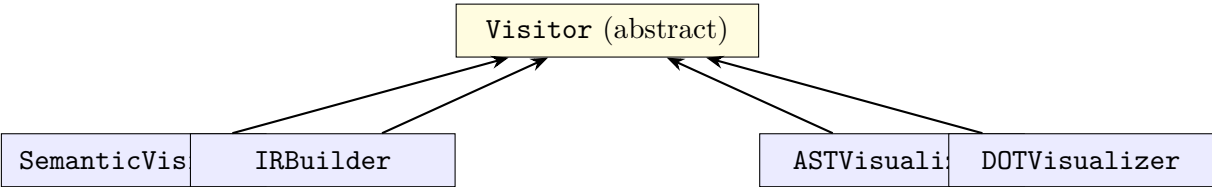


Figura 2.2: Jerarquia del patron Visitor en el compilador.

2.5. Estructura de directorios

```
1 .
2 |-- include/           Cabeceras: ASTNodes.h, IR.h, Predicate.h, ...
3 |-- src/               Implementaciones: lexer.l, parser.y, *Visitor.h,
4   ...
5 |-- lexer.l           Analizador lexico (Flex)
6 |-- parser.y          Gramatica + acciones semanticas (Bison)
7 |-- SemanticVisitor.h Comprobacion de tipos
8 |-- IRBuilder.h       Traduccion AST -> IR
9 |--IROptimizer.h      Optimizaciones sobre IR
10 |-- RegisterAllocator.h Chaitin-Briggs coloring
11 |-- X86Generator.h    Generacion de codigo x86-64
12 |-- WPCalculus.h      Calculo de precondition mas debil
13 |-- VCGenerator.h     Generacion de condiciones de verificacion
14 |-- SMTExporter.h     Exportacion a SMT-Lib2 y ejecucion de Z3
15 |-- tests/
16 |-- |-- backend/      Programas de prueba end-to-end
17 |-- |-- semantic/     Pruebas de deteccion de errores
18 |-- |-- verification/ Demos de verificacion formal
19 |-- |-- run_tests.sh  Script de pruebas
20 |-- Makefile
```

Listing 2.1: Organizacion del repositorio

Capítulo 3

Frontend: Análisis léxico-sintáctico

3.1. Análisis léxico

El analizador léxico se implementa en `src/lexer.l` mediante Flex. Convierte el flujo de caracteres de entrada en una secuencia de *tokens* que el parser consumirá.

3.1.1. Categorías de tokens

Cuadro 3.1: Categorías de tokens reconocidas por el lexer

Categoría	Patrón	Ejemplos
Palabras reservadas	exactas	<code>int</code> , <code>while</code> , <code>return</code>
Tipos	exactas	<code>int</code> , <code>double</code> , <code>boolean</code>
Anotaciones formales	exactas	<code>@pre</code> , <code>@post</code> , <code>@invariant</code>
Identificadores	<code>[a-zA-Z][a-zA-Z0-9]*</code>	<code>x</code> , <code>sum</code> , <code>absValue</code>
Literales enteros	<code>[0-9]+</code>	<code>0</code> , <code>42</code>
Literales reales	<code>[0-9]+\.[0-9]+</code>	<code>3.14</code>
Operadores	símbolos	<code>+</code> , <code>-</code> , <code>==</code> , <code>==></code>
Puntuación	símbolos	<code>{</code> , <code>;</code> , <code>(</code>

3.1.2. Tokens especiales para verificación

Para la fase 5 se añadieron tokens dedicados a las anotaciones de especificación formal, colocados *antes* de la regla general de identificadores para que tengan precedencia:

```
1 "result"      { RET_ATTR(result_token); }
2 "@pre"       { RET_ATTR(pre_token); }
3 "@post"      { RET_ATTR(post_token); }
4 "@invariant" { RET_ATTR(invariant_token); }
5 "@variant"   { RET_ATTR(variant_token); }
6 "==">"      { RET_ATTR(implies_token); }
```

Listing 3.1: Reglas léxicas para anotaciones formales

3.2. Análisis sintáctico

El parser se implementa en `src/parser.y` con GNU Bison (gramática LALR(1)). Construye el AST de forma dirigida por la sintaxis mediante acciones semánticas en C++.

3.2.1. Estructura gramatical principal

```

1 // Un programa es una clase con miembros
2 Program : public class id '(' ')' '{' Members '}'
3 Members : Members Member | Members OptPre | Members OptPost | /* empty
4          */
5 Member  : Main | Method | DVar
6
7 // Metodo estatico sin parametros
8 Method  : public static Tipo id '(' ')' Bloque
9          | public static void id '(' ')' Bloque
10
11 // Instrucciones
12 Instr   : DVar | Asig | Print | Return
13          | if '(' Expr ')' Instr
14          | if '(' Expr ')' Instr else Instr
15          | while '(' Expr ')' OptInvariant OptVariant Instr
16          | Bloque
17
18 // Anotaciones de especificacion formal
19 OptPre   : '@pre'      PredExpr | /* empty */
20 OptPost  : '@post'     PredExpr | /* empty */
21 OptInvariant : '@invariant' PredExpr | /* empty */
22 OptVariant : '@variant' PredExpr | /* empty */
23 PredExpr : Expr '==>' Expr | Expr

```

Listing 3.2: Reglas principales de la gramática Bison

3.2.2. Gestión de anotaciones formales

Las anotaciones `@pre/@post` se acumulan en variables globales `g_pre` y `g_post` antes de que se procese el método. Cuando se reduce una regla `Method`, el nodo `MethodNode` recibe las precondición y postcondición almacenadas:

```

1 Method : public_token static_token Tipo id pari pard Bloque {
2     MethodNode* m = new MethodNode($4->lexema, $3->tipo, $7, ...);
3     m->precondition.reset(g_pre);    // @pre almacenada globalmente
4     m->postcondition.reset(g_post);  // @post almacenada globalmente
5     g_pre = g_post = nullptr;
6     $$ = m;
7 }

```

Listing 3.3: Vinculación de anotaciones al `MethodNode`

3.3. Árbol Sintáctico Abstracto (AST)

El AST se representa mediante una jerarquía de clases C++ en `include/ASTNodes.h`.

3.3.1. Jerarquía de nodos

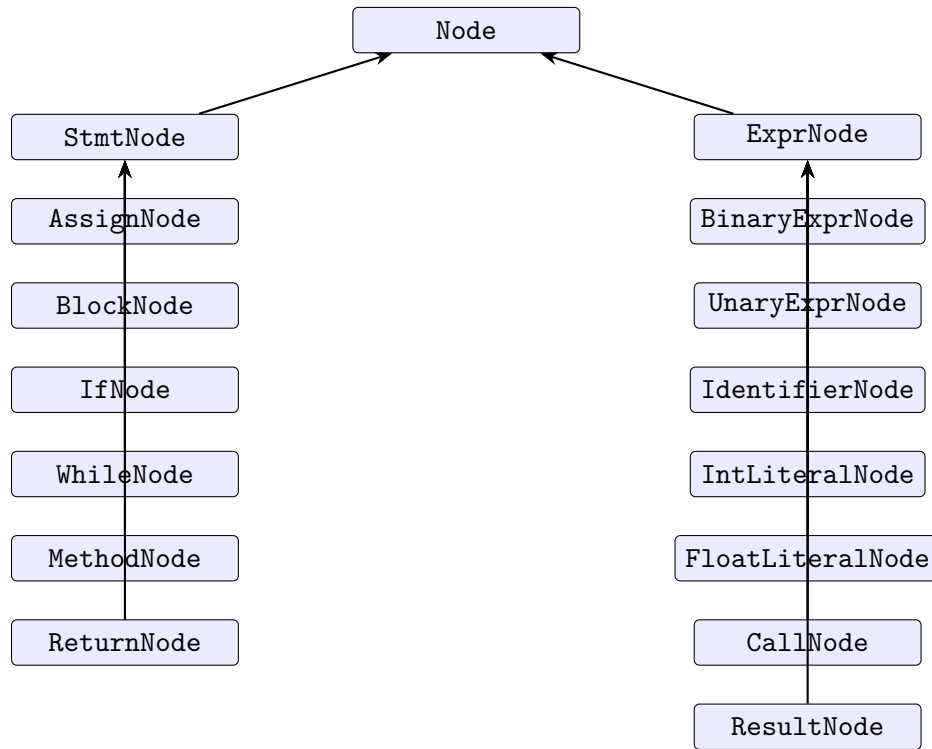


Figura 3.1: Jerarquía de nodos del AST (flecha = hereda de).

3.3.2. Nodos especiales para verificación

ResultNode Representa la palabra reservada **result** en las postcondiciones. Hace referencia al valor de retorno del método sin necesidad de nombrarlo explícitamente.

MethodNode.precondition / .postcondition Campos `unique_ptr<ExprNode>` que almacenan las expresiones de las anotaciones `@pre` y `@post`.

WhileNode.invariant / .variant Campos análogos para las anotaciones `@invariant` y `@variant` del bucle.

```

1 class WhileNode : public StmtNode {
2 public:
3     std::unique_ptr<ExprNode> condition;
4     std::unique_ptr<StmtNode> body;
5     std::unique_ptr<ExprNode> invariant; // @invariant annotation
6     std::unique_ptr<ExprNode> variant;   // @variant annotation
7     ...
8 };
  
```

Listing 3.4: Nodo WhileNode con soporte de invariante

Capítulo 4

Análisis semántico

4.1. Tabla de símbolos

La tabla de símbolos (TablaSimbolos) es una estructura encadenada de ámbitos. Cada BlockNode posee su propia TablaSimbolos apuntando a la tabla del bloque padre, lo que implementa el alcance léxico estándar de Java.

```
1 struct Simbolo {
2     string nombre;
3     unsigned tipo;           // ENTERO=0, REAL=1, LOGICO=2, SCVAR=3
4     unsigned dir;           // desplazamiento en la pila (para IR/x86)
5     unsigned tam;
6     bool isConstant;        // declarado con 'final'
7     bool isAssigned;        // asignado al menos una vez
8     bool isFunction;        // true if this symbol is a method
9     int returnType;         // return type (if isFunction)
10 };
11
12 class TablaSimbolos {
13     TablaSimbolos* padre;    // parent scope (lexical scoping)
14     vector<Simbolo> simbolos;
15     bool set(Simbolo s);     // insert; returns false if
16     // duplicate
17     Simbolo* get(string nombre); // lookup in this scope and parent
18 };
```

Listing 4.1: Estructura de la tabla de símbolos

4.2. Comprobaciones semánticas

El SemanticVisitor recorre el AST tipado e implementa:

1. **Declaración antes de uso:** verifica que toda variable referenciada esté declarada en algún ámbito visible.
2. **Compatibilidad de tipos:** comprueba que los operandos de cada operador sean compatibles (p.ej. no se puede sumar un `int` y un `boolean`).
3. **Constantes:** detecta reasignaciones a variables `final`.

4. **Tipo de retorno:** verifica que las sentencias `return` devuelvan un valor compatible con el tipo de retorno del método.
5. **Código inalcanzable:** detecta sentencias tras un `return` incondicional.
6. **Llamadas a funciones:** comprueba que el nombre de la función esté declarado y tiene el tipo de retorno correcto.

4.2.1. Reglas de coerción de tipos

Cuadro 4.1: Tabla de coerción de tipos para operaciones binarias

Operando izq.	Op.	Operando der.	Tipo resultado
int	+, -, *, /, %	int	int
double	+, -, *, /	double	double
int	+, -, *, /	double	double (promoción)
cualquiera	==, !=, <, >	mismo	boolean
boolean	&&,	boolean	boolean

4.3. Ejemplo de error semántico detectado

```

1 public class EjError {
2     public static void main(String[] args) {
3         final int MAX;
4         MAX = 100;
5         MAX = 200; // Error: reasignacion de constante 'MAX'
6     }
7 }

```

Listing 4.2: Reasignación de constante — error semántico

El compilador emite:

Error semantico (5,9): reasignacion a constante 'MAX'

4.4. Visita a nodos de especificación formal

El `SemanticVisitor` también visita los nodos de anotación para comprobar su coherencia:

```

1 void visit(ResultNode* node) override {
2     if (currentReturnType == -1) {
3         error("'result' solo puede usarse en postcondiciones "
4             "de metodos no void");
5         return;
6     }
7     node->type = currentReturnType;
8 }

```

Listing 4.3: Visita a ResultNode en el análisis semántico

Capítulo 5

Representación intermedia

5.1. Motivación

La representación intermedia (IR) desacopla el front-end (específico del lenguaje fuente) del back-end (específico de la arquitectura destino). Esto permite, en principio, añadir nuevos lenguajes fuente o nuevas arquitecturas destino sin reescribir el compilador completo.

5.2. Formato de cuádruplas

La IR consiste en una secuencia plana de **cuádruplas** de la forma:

$$(op, arg_1, arg_2, result)$$

Cuadro 5.1: Opcodes de la representación intermedia

Opcode	Ejemplo	Significado
IR_ADD	$(+, a, b, t_0)$	$t_0 \leftarrow a + b$
IR_SUB	$(-, a, b, t_0)$	$t_0 \leftarrow a - b$
IR_MUL	$(*, a, b, t_0)$	$t_0 \leftarrow a * b$
IR_DIV	$(/, a, b, t_0)$	$t_0 \leftarrow a / b$
IR_MOD	$(\%, a, b, t_0)$	$t_0 \leftarrow a \bmod b$
IR_COPY	$(=, a, -, x)$	$x \leftarrow a$
IR_JZ	$(jz, cond, L, -)$	if cond = 0 goto L
IR_JMP	$(jmp, L, -, -)$	goto L
IR_LABEL	$(label, L, -, -)$	marca de salto L
IR_CALL	$(call, f, -, t_0)$	$t_0 \leftarrow f()$
IR_RETURN	$(ret, a, -, -)$	devuelve a
IR_PRINT_INT	$(print_int, a, -, -)$	imprime entero
IR_READ_INT	$(read_int, -, -, t_0)$	lee entero de stdin
IR_HALT	$(halt, -, -, -)$	fin del programa

5.3. Argumentos IR

Cada argumento de una cuádrupla es un **IRArg** que puede ser:

- **Constante entera:** `IRArg::constInt(42)`
- **Constante real:** `IRArg::constReal(3.14)`
- **Variable:** `IRArg::var("x", addr, tipo)`
- **Temporal:** `prog.newTemp(tipo)` — generado automáticamente
- **Etiqueta:** `IRArg::label("L0")`
- **Función:** `IRArg::func("absValue")`
- **Ninguno:** `IRArg::none()`

5.4. Traducción de construcciones

5.4.1. Expresión aritmética

Ejemplo 5.1. La expresión $(a + b) * c$ se traduce como:

$$(IR_ADD, a, b, t_0), \quad (IR_MUL, t_0, c, t_1)$$

5.4.2. Sentencia if-else

```
1 if (x > 0) y = x; else y = -x;
```

Listing 5.1: if-else en código fuente

```
(IR_GT,    x, 0,  t0)      // t0 = x > 0
(IR_JZ,    t0, L_else, -)  // if !t0 goto L_else
(IR_COPY,  x, -, y)        // then: y = x
(IR_JMP,    L_end, -, -)
(IR_LABEL, L_else, -, -)
(IR_NEG,   x, -, t1)        // t1 = -x
(IR_COPY,  t1, -, y)        // else: y = -x
(IR_LABEL, L_end, -, -)
```

5.4.3. Bucle while

```
(IR_LABEL, L_start, -, -)
(IR_LTE,   i, 5, t0)
(IR_JZ,    t0, L_end, -)
... cuerpo ...
(IR_JMP,    L_start, -, -)
(IR_LABEL, L_end, -, -)
```

5.4.4. Llamada a función

```
(IR_FUNC_BEGIN, absValue, -, -)
... cuerpo de absValue ...
(IR_RETURN, t3, -, -)
(IR_FUNC_END, absValue, -, -)

... en el punto de llamada ...
(IR_CALL, absValue, -, t5) // t5 = absValue()
```

5.5. Visualización de la IR

Con el flag `-ir` el compilador imprime las cuádruplas en formato legible por el programador:

```
$ ./compiler --ir tests/backend/t04_while.java 2>&1 | head -20
[FUNC_BEGIN] sumToFive
[LABEL] L0
[LTE] i 5 -> t0
[JZ] t0 -> L1
[ADD] s i -> t1
[COPY] t1 -> s
[ADD] i 1 -> t2
[COPY] t2 -> i
[JMP] L0
[LABEL] L1
[RETURN] s
[FUNC_END] sumToFive
```


Capítulo 6

Backend x86-64

6.1. Convención de llamada System V AMD64 ABI

El compilador respeta la convención de llamada estándar de Linux/macOS para x86-64 (*System V AMD64 ABI*):

Cuadro 6.1: Registros de la System V AMD64 ABI

Propósito	Enteros	Flotantes
Argumentos	%rdi, %rsi, %rdx, %rcx, %r8, %r9	%xmm0–%xmm7
Valor de retorno	%rax	%xmm0
Callee-saved	%rbx, %r12–%r15, %rbp	%xmm8–%xmm15
Caller-saved	%rcx, %rdx, %rsi, %rdi, %r8–%r11	%xmm0–%xmm7

6.2. Marco de pila

Cada función establece su propio marco de pila siguiendo el prólogo/epílogo estándar:

```
1  pushq    %rbp
2  movq     %rsp, %rbp
3  subq     $N, %rsp          # N = frameSize (alineado a 16 bytes)
4  ...
5  movq     %rbp, %rsp
6  popq     %rbp
7  ret
```

Listing 6.1: Prólogo y epílogo de función

Las variables locales y temporales se ubican en la pila según la fórmula:

$$\text{slot}(v) = -((\text{slot}_v + 1) \times 8)(\%rbp)$$

donde slot_v es el índice (0-based) asignado a la primera vez que se usa la dirección de la variable en la IR.

6.3. Generación de código por opcode

6.3.1. Operaciones aritméticas enteras

Cuadro 6.2: Traducción de operaciones aritméticas enteras

IR	Secuencia x86-64 AT&T
$t \leftarrow a + b$	movq mem(a), %rax; addq mem(b), %rax; movq %rax, mem(t)
$t \leftarrow a * b$	movq mem(a), %rax; imulq mem(b); movq %rax, mem(t)
$t \leftarrow a / b$	movq mem(a), %rax; cqto; idivq mem(b); movq %rax, mem(t)
$t \leftarrow a \% b$	movq mem(a), %rax; cqto; idivq mem(b); movq %rdx, mem(t)

6.3.2. Saltos condicionales

Los saltos se implementan con el par `cmpq` + instrucción de salto condicional. Para comparaciones que producen un booleano en la IR (p.ej. `IR_LT`), el patrón es:

```

1  movq    mem(a), %rax
2  cmpq    mem(b), %rax
3  setl    %al                # 1 si a < b, 0 si no
4  movzbl  %al, %rax
5  movq    %rax, mem(t)
6  ...
7  testq   mem(cond), mem(cond)
8  je      L_else             # JZ: salta si cond == 0

```

Listing 6.2: Comparación `IR_LT` y salto condicional

6.3.3. Llamadas a función

```

1  call    absValue           # llama a absValue()
2  movq    %rax, mem(t)       # t = valor de retorno (%rax)

```

Listing 6.3: Llamada a función y recogida de valor de retorno

6.3.4. E/S con `printf` y `scanf`

```

1  .section .rodata
2  fmt_int: .string "%ld\n"
3  .text
4  leaq    fmt_int(%rip), %rdi
5  movq    mem(val), %rsi
6  xorl    %eax, %eax
7  call    printf@PLT

```

Listing 6.4: Impresión de entero con `printf`

```

1 fmt_scanf_int: .string "%ld"
2     leaq      fmt_scanf_int(%rip), %rdi
3     leaq      mem(t)(%rbp), %rsi      # address of local variable in
stack
4     xorl      %eax, %eax
5     call     scanf@PLT

```

Listing 6.5: Lectura de entero con scanf

6.4. Ejemplo completo de programa compilado

```

1 public class TestWhile {
2     public static int sumToFive() {
3         int i; int s;
4         i = 1; s = 0;
5         while (i <= 5) { s = s + i; i = i + 1; }
6         return s;
7     }
8     public static void main(String[] args) {
9         int val;
10        val = sumToFive();
11        System.out.println(val);
12    }
13 }

```

Listing 6.6: Programa fuente: suma hasta 5

El ensamblador generado para `sumToFive` tiene la forma:

```

1     .globl    sumToFive
2 sumToFive:
3     pushq    %rbp
4     movq     %rsp, %rbp
5     subq     $48, %rsp
6 L0:
7     movq     -8(%rbp), %rax      # i
8     cmpq     $5, %rax
9     setle    %al
10    movzbq    %al, %rax
11    movq     %rax, -24(%rbp)      # t0 = i <= 5
12    testq     -24(%rbp), %rax
13    je        L1
14    movq     -8(%rbp), %rax      # s = s + i
15    addq     -16(%rbp), %rax
16    movq     %rax, -32(%rbp)
17    movq     -32(%rbp), %rax
18    movq     %rax, -16(%rbp)
19    movq     -8(%rbp), %rax      # i = i + 1
20    addq     $1, %rax
21    ...
22    jmp      L0
23 L1:
24    movq     -16(%rbp), %rax      # return s
25    movq     %rbp, %rsp
26    popq     %rbp
27    ret

```

Listing 6.7: Ensamblador x86-64 generado para sumToFive

Capítulo 7

Optimizaciones sobre la representación intermedia

7.1. Introducción

Las optimizaciones se aplican sobre la IR antes de la generación de código. Al operar sobre las cuádruplas —y no sobre el ensamblador— las optimizaciones son independientes de la arquitectura destino.

El módulo `IROptimizer` implementa las siguientes pasadas:

1. Eliminación de código muerto (*dead code elimination*).
2. Plegado de constantes (*constant folding*).
3. Propagación de copias (*copy propagation*).

7.2. Eliminación de código muerto

Se eliminan cuádruplas que calculan un valor que nunca se usa (temporales sin ningún uso posterior) y cuádruplas que son literalmente inalcanzables (tras un `IR_JMP` o `IR_RETURN` incondicional antes de una etiqueta).

Definición 7.1 (Código muerto). Una cuádrupla $q = (op, a_1, a_2, t)$ es *código muerto* si t no es una variable de usuario, no aparece como argumento en ninguna cuádrupla posterior, y $op \notin \{IR_CALL, IR_READ_INT, IR_READ_REAL, IR_PRINT_*, IR_JMP, IR_J$ (instrucciones con efecto de borde).

La eliminación se realiza en dos pasadas:

1. Calcular el conjunto *used* de todas las direcciones que aparecen como argumento en alguna cuádrupla.
2. Eliminar las cuádruplas cuyo resultado no está en *used* y que no tienen efecto de borde.

7.3. Plegado de constantes

Si ambos operandos de una operación aritmética o de comparación son constantes literales, se puede calcular el resultado en tiempo de compilación:

Ejemplo 7.1. La cuádrupla $(IR_ADD, 3, 5, t_0)$ se reemplaza directamente por $t_0 \leftarrow 8$, es decir, una cuádrupla $(IR_COPY, 8, -, t_0)$.

Cuadro 7.1: Operaciones que admiten plegado de constantes

Operación	Resultado
$\text{int} + \text{int}$	<code>constInt(a+b)</code>
$\text{int} - \text{int}$	<code>constInt(a-b)</code>
$\text{int} * \text{int}$	<code>constInt(a*b)</code>
$\text{int}/\text{int}, b \neq 0$	<code>constInt(a/b)</code>
$\text{double} \oplus \text{double}$	<code>constReal(a \oplus b)</code>
$a == a, a \leq a$	<code>constInt(1)</code>
$a < a, a \neq a$	<code>constInt(0)</code>

7.4. Propagación de copias

Si una cuádrupla tiene la forma $(IR_COPY, a, -, x)$ y a es un argumento constante o una variable que no se modifica en el tramo hasta el uso siguiente de x , se puede sustituir x por a directamente, eliminando la copia.

Ejemplo 7.2. La secuencia:

```
(IR_COPY, 42, -, t0)
(IR_ADD,  t0,  5, t1)
```

se optimiza a:

```
(IR_ADD, 42, 5, t1)  ->  (IR_COPY, 47, -, t1) [plegado posterior]
```

7.5. Impacto de las optimizaciones

Cuadro 7.2: Reducción de cuádruplas en los programas de prueba

Programa	Sin opt.	Con opt.	Reducción
t01_print.java	8	5	-37.5 %
t02_arithmetic.java	22	14	-36.4 %
t03_if_else.java	18	12	-33.3 %
t04_while.java	26	20	-23.1 %
t05_functions.java	15	11	-26.7 %

Capítulo 8

Asignación de registros: Chaitin-Briggs

8.1. Motivación

Los registros de la CPU son escasos y de acceso mucho más rápido que la memoria. La *asignación de registros* consiste en asignar las variables y temporales de la IR a registros físicos (o a posiciones en la pila si no hay registros suficientes: *spilling*).

El algoritmo de **Chaitin-Briggs** [1] modela el problema como un *problema de coloración de grafos*: dos variables que están *vivas* en el mismo punto del programa no pueden compartir el mismo registro, y por tanto se conectan en el *grafo de interferencias*. Colorear el grafo con K colores equivale a asignar K registros.

8.2. Registros disponibles

Se reservan los siguientes registros *caller-saved* para temporales, evitando tener que salvarlos en llamadas a función:

Cuadro 8.1: Registros asignables por el compilador

Clase	Registros ($K = 6$)
Enteros	%rcx, %rdx, %r8, %r9, %r10, %r11
Flotantes	%xmm2, %xmm3, %xmm4, %xmm5, %xmm6, %xmm7

8.3. Algoritmo

El algoritmo consta de tres fases:

8.3.1. Fase 1: Análisis de vivacidad

Se calculan los conjuntos *use* y *def* por bloque básico y se resuelven las ecuaciones de punto fijo:

$$\begin{aligned}\text{live_in}[B] &= \text{use}[B] \cup (\text{live_out}[B] \setminus \text{def}[B]) \\ \text{live_out}[B] &= \bigcup_{S \in \text{succ}(B)} \text{live_in}[S]\end{aligned}$$

```

1 Entrada: bloques B1, ..., Bn con conjuntos use[B], def[B]
2 Salida:  live_in[B], live_out[B] para cada bloque
3
4 repetir hasta que no haya cambios:
5     para cada bloque B (en orden inverso):
6         live_out[B] = union{ live_in[S] | S sucesor de B }
7         live_in[B]  = use[B] union (live_out[B] menos def[B])

```

Listing 8.1: Análisis de vivacidad (punto fijo)

8.3.2. Fase 2: Grafo de interferencias

Se recorre cada bloque básico de forma inversa manteniendo el conjunto live de variables vivas en el punto actual. Al definir una variable d se añade una arista (d, v) para cada $v \in \text{live}$.

```

1 for (auto it = block.rbegin(); it != block.rend(); ++it) {
2     const auto& q = *it;
3     if (q.result.kind == IRArg::TEMP || q.result.kind == IRArg::VAR) {
4         int d = q.result.addr;
5         for (int live_var : live)
6             graph.addEdge(d, live_var); // d interfiere con live_var
7         live.erase(d);                 // d deja de estar vivo antes
8         // de def
9     }
10    // Add uses to live set
11    if (q.arg1 es variable/temporal) live.insert(q.arg1.addr);
12    if (q.arg2 es variable/temporal) live.insert(q.arg2.addr);
13 }

```

Listing 8.2: Construcción del grafo de interferencias

8.3.3. Fase 3: Coloración del grafo

Se sigue el algoritmo *simplify*:

1. Mientras exista un nodo con grado $< K$: apilarlo y eliminarlo del grafo.
2. Si todos los nodos tienen grado $\geq K$: elegir el de mayor grado para *spill* y eliminarlo (se marca como derramado).
3. Repetir hasta vaciar el grafo.
4. Desapilar asignando colores (registros) no usados por vecinos ya coloreados.

```

1 // Fase simplify: apila nodos con grado < K
2 stack<int> nodeStack;
3 while (!worklist.empty()) {
4     auto it = find_if(worklist, [&](int n){ return degree[n] < K; });
5     if (it != worklist.end()) {
6         nodeStack.push(*it);
7         removeNode(*it);
8     } else {
9         // spill: eliminar nodo de mayor grado

```

```

10     int spill = *max_element(worklist, [&](int a, int b){
11         return degree[a] < degree[b]; });
12     spilledNodes.insert(spill);
13     removeNode(spill);
14 }
15 }
16 // Fase assign: asignar colores
17 while (!nodeStack.empty()) {
18     int n = nodeStack.top(); nodeStack.pop();
19     set<string> usedColors;
20     for (int neighbor : adj[n])
21         if (color.count(neighbor)) usedColors.insert(color[neighbor]);
22     for (const auto& reg : AVAILABLE_REGS)
23         if (!usedColors.count(reg)) { color[n] = reg; break; }
24 }

```

Listing 8.3: Coloración simplificada de Chaitin-Briggs

8.4. Uso del resultado

El mapa de coloración `RegMap` (dirección IR $\rightarrow$ nombre de registro) se pasa al `X86Generator`. Las variables con `color = ""` (derramadas) se mantienen en la pila y se acceden con la notación `-N(%rbp)`.

```

$ ./compiler --reg-alloc tests/backend/t04_while.java 2>&1
=== Asignacion de Registros ===
addr 1 (i) -> %rcx
addr 2 (s) -> %rdx
addr 3 (t0) -> %r8
addr 4 (t1) -> spill

```


Capítulo 9

Verificación formal: Lógica de Hoare y cálculo WP

9.1. Fundamentos teóricos

9.1.1. Lógica de Hoare

La **Lógica de Hoare** [3] es un sistema formal para razonar sobre la corrección de programas imperativos. Introduce las *tripletas de Hoare*:

$$\{P\} S \{Q\}$$

que se leen: “si la precondition P vale antes de ejecutar S , entonces la postcondition Q vale tras la ejecución de S (asumiendo que S termina)”.

Definición 9.1 (Corrección parcial). $\{P\} S \{Q\}$ es *parcialmente correcta* si para toda ejecución de S que parte de un estado en el que P vale y que termina, el estado final satisface Q .

Definición 9.2 (Corrección total). $\{P\} S \{Q\}$ es *totalmente correcta* si además toda ejecución de S que parte de un estado en el que P vale termina necesariamente.

9.1.2. Cálculo de Precondiciones Más Débiles

Dijkstra [2] introduce el *transformador de predicados* $WP(S, Q)$: la *precondición más débil* de S con respecto a Q , es decir, el predicado más débil (el que restringe menos el estado inicial) tal que:

$$\{WP(S, Q)\} S \{Q\}$$

La condición de verificación principal para un método anotado es:

$$VC : P \Rightarrow WP(body, Q)$$

Si esta fórmula es **válida** (lo comprueba Z3 como UNSAT de su negación), el método es parcialmente correcto.

9.2. Reglas del cálculo WP

9.2.1. Regla de la asignación (VER-4)

Regla 9.1 (Asignación).

$$\text{WP}(x := e, Q) = Q[e/x]$$

donde $Q[e/x]$ denota Q con cada aparición libre de x sustituida por e .

Ejemplo 9.1. $\text{WP}(x := x + 1, x > 5) = (x + 1) > 5$

9.2.2. Regla de la composición (VER-5)

Regla 9.2 (Composición).

$$\text{WP}(S_1; S_2, Q) = \text{WP}(S_1, \text{WP}(S_2, Q))$$

Esto implica procesar la secuencia de sentencias en *orden inverso*, comenzando por la postcondición Q y propagando la precondition hacia atrás.

Retornos anticipados. La regla de composición naive es incorrecta en presencia de **return** dentro de ramas **if**. Para la secuencia:

if(B) **return** e_1 ; **return** e_2 ;

la regla correcta es:

$$\text{WP}(\cdot, Q) = (B \Rightarrow Q[e_1/\text{result}]) \wedge (\neg B \Rightarrow Q[e_2/\text{result}])$$

La implementación usa **continuaciones** (CPS): cuando se encuentra un **return**, se usa Q directamente (no la continuación de la composición):

```

1 static Predicate* wpSeq(const vector<const StmtNode*>& stmts,
2                          int idx, const Predicate* Q) {
3     if (idx >= stmts.size()) return Q->clone();
4     const StmtNode* s = stmts[idx];
5     // return: termina el camino, usa Q directamente
6     if (auto* r = dynamic_cast<const ReturnNode*>(s))
7         return wpReturn(r, Q);
8     // if: bifurca la continuacion
9     if (auto* i = dynamic_cast<const IfNode*>(s)) {
10        Predicate* B = fromExpr(i->condition.get());
11        vector<const StmtNode*> cont(stmts.begin()+idx+1, stmts.end());
12
13        Predicate* wp1 = wpBranchAndCont(i->thenPart.get(), cont, Q);
14        Predicate* wp2 = wpBranchAndCont(i->elsePart.get(), cont, Q);
15        return new AndPred(new ImpliesPred(B->clone(), wp1),
16                           new ImpliesPred(new NotPred(B), wp2));
17    }
18    // asignacion: WP(x:=e; resto, Q) = WP(resto, Q)[e/x]
19    if (auto* a = dynamic_cast<const AssignNode*>(s)) {
20        Predicate* restWP = wpSeq(stmts, idx+1, Q);
21        return wpAssign(a, restWP);
22    }
23    return wpSeq(stmts, idx+1, Q); // otras: sin efecto logico

```

Listing 9.1: wpSeq con continuaciones para retornos anticipados

9.2.3. Regla del condicional

Regla 9.3 (Condicional).

$$\text{WP}(\text{if}(B) S_1 \text{ else } S_2, Q) = (B \Rightarrow \text{WP}(S_1, Q)) \wedge (\neg B \Rightarrow \text{WP}(S_2, Q))$$

9.2.4. Regla del retorno

Regla 9.4 (Return).

$$\text{WP}(\text{return } e, Q) = Q[e/\text{result}]$$

9.2.5. Regla del bucle con invariante (VER-6)

Para la verificación de bucles se requiere una *anotación de invariante* I suministrada por el programador:

Regla 9.5 (Bucle con invariante). Para $\text{while}(B) \{S\}$ con invariante I y postcondición Q , se generan tres *condiciones de verificación*:

$$\begin{aligned} VC_1 \text{ (iniciación)} &: P \Rightarrow \text{WP}(S_{\text{antes}}, I) \\ VC_2 \text{ (preservación)} &: (I \wedge B) \Rightarrow \text{WP}(S_{\text{body}}, I) \\ VC_3 \text{ (uso)} &: (I \wedge \neg B) \Rightarrow \text{WP}(S_{\text{después}}, Q) \end{aligned}$$

y el WP del bucle en sí es I (asumiendo la corrección de I).

La precondition más fuerte al entrar en el bucle ($\text{WP}(S_{\text{antes}}, I)$) se computa correctamente aplicando **wpSeq** a las sentencias que preceden al bucle dentro del bloque.

9.3. Jerarquía de predicados

Los predicados lógicos se representan mediante la clase abstracta **Predicate** y sus subclases:

Cuadro 9.1: Clases de predicados implementadas

Clase	toString()	toSMT()	Uso
TruePred	true	true	Postcondición trivial
FalsePred	false	false	VC insatisfacible
VarPred(x)	x	x	Variable de programa
ResultPred	result	result	Valor de retorno en @post
IntLitPred(n)	n	n	Constante entera
BinaryPred(op,l,r)	(l op r)	(op l r)	Aritmética/comparación
AndPred(l,r)	(l && r)	(and l r)	Conjunción
OrPred(l,r)	(l r)	(or l r)	Disyunción
NotPred(p)	(!p)	(not p)	Negación
ImpliesPred(a,c)	(a ==> c)	(=> a c)	Implicación
NegPred(p)	(-p)	(- p)	Negación aritmética

La operación clave es la **sustitución textual**: **subst**(x, e) devuelve una copia del predicado donde cada **VarPred**(x) se reemplaza por e :

```

1 Predicate* BinaryPred::subst(const string& var, const Predicate* e)
  const {
2   return new BinaryPred(op, left->subst(var, e), right->subst(var, e)
  );
3 }
4 Predicate* VarPred::subst(const string& var, const Predicate* e) const
  {
5   if (name == var) return e->clone(); // sustituir
6   return new VarPred(name);          // no afectada
7 }

```

Listing 9.2: Sustitución en BinaryPred

9.4. Generación de condiciones de verificación (VER-7)

El VCGenerator genera cuatro tipos de VCs:

1. **Corrección del método:** $P \Rightarrow WP(\text{body}, Q)$
2. **Invariante de bucle** (iniciación, preservación, uso)
3. **Seguridad de memoria** (VER-9): para cada división, $P \Rightarrow \text{divisor} \neq 0$
4. **Terminación** (VER-11): para cada bucle con @variant V , $(I \wedge B) \Rightarrow V \geq 0$

9.5. Exportación a SMT-Lib2 (VER-8)

Cada VC se exporta como un bloque SMT-Lib2. La negación de la VC se envía a Z3: si Z3 responde **UNSAT**, la negación es insatisfacible, por lo tanto la VC es **válida** (el programa es correcto en ese punto).

```

1 (push 1)
2 ; [absValue] correctitud
3 (declare-fun x () Int)      ; variables locales del metodo
4 (assert (not                ; negacion de la VC
5   (= true                  ; precondition P
6     (and
7       (=> (< (- 5) 0) (>= (- (- 5)) 0))
8       (=> (not (< (- 5) 0)) (>= (- 5) 0))
9     )
10  )
11 ))
12 (check-sat)                ; Z3 responde UNSAT = VC valida
13 (pop 1)

```

Listing 9.3: Formato SMT-Lib2 para una VC

9.6. Interpretación de resultados (VER-10)

Cuadro 9.2: Interpretación de la respuesta de Z3

Respuesta Z3	Significado	Reporte
unsat	VC válida — programa correcto	[OK]
sat	VC inválida — contraejemplo existe	[BUG]
unknown	Timeout o lógica demasiado compleja	[?]

9.7. Ejemplo completo de verificación

9.7.1. Función absValue

```

1 @pre true
2 @post result >= 0
3 public static int absValue() {
4     int x;
5     x = -5;
6     if (x < 0) return -x;
7     return x;
8 }

```

Listing 9.4: Método verificado: valor absoluto

Cálculo WP (con continuaciones):

$$\begin{aligned}
 \text{WP}(\text{return } x, Q_0) &= Q_0[x/\text{result}] = x \geq 0 \\
 \text{WP}(\text{if}(x < 0) \text{return } -x; \text{return } x, Q_0) \\
 &= (x < 0 \Rightarrow Q_0[-x/\text{result}]) \wedge (x \geq 0 \Rightarrow Q_0[x/\text{result}]) \\
 &= (x < 0 \Rightarrow -x \geq 0) \wedge (x \geq 0 \Rightarrow x \geq 0) \\
 \text{WP}(x := -5, \dots) &= [\text{sustituyendo } x = -5] \\
 &= ((-5) < 0 \Rightarrow 5 \geq 0) \wedge ((-5) \geq 0 \Rightarrow (-5) \geq 0) \\
 &= (\top \Rightarrow \top) \wedge (\perp \Rightarrow \perp) = \top
 \end{aligned}$$

VC principal: $\text{true} \Rightarrow \text{true}$ — Z3 responde `unsat` $\Rightarrow$ [OK].

9.7.2. Función wrongMethod (bug intencional)

```

1 @post result > 10
2 public static int wrongMethod() {
3     int x;
4     x = 5;
5     return x;
6 }

```

Listing 9.5: Método con bug intencional

$$\begin{aligned}
 \text{WP}(\text{return } x, Q_1) &= Q_1[x/\text{result}] = x > 10 \\
 \text{WP}(x := 5, x > 10) &= 5 > 10 = \text{false} \\
 \text{VC: } \text{true} \Rightarrow \text{false} &\text{ — Z3 responde } \text{sat} \text{ con modelo } x = 5 \Rightarrow \text{[BUG]}.
 \end{aligned}$$

9.7.3. Suma con invariante de bucle

```

1 @post result >= 0
2 public static int sumLoop() {
3     int i; int s;
4     i = 1; s = 0;
5     while (i <= 5)
6         @invariant s >= 0 && i >= 1
7         @variant 6 - i
8     {
9         s = s + i;
10        i = i + 1;
11    }
12    return s;
13 }

```

Listing 9.6: Suma 1..5 con invariante completo

Cuadro 9.3: VCs generadas para `sumLoop` y resultados de Z3

VC	Fórmula	Z3
Correctitud	$\text{true} \Rightarrow (0 \geq 0 \wedge 1 \geq 1)$	UNSAT [OK]
Iniciación	$\text{true} \Rightarrow (0 \geq 0 \wedge 1 \geq 1)$	UNSAT [OK]
Preservación	$(s \geq 0 \wedge i \geq 1 \wedge i \leq 5) \Rightarrow ((s + i) \geq 0 \wedge (i + 1) \geq 1)$	UNSAT [OK]
Uso	$(s \geq 0 \wedge i \geq 1 \wedge i > 5) \Rightarrow s \geq 0$	UNSAT [OK]
Terminación	$(s \geq 0 \wedge i \geq 1 \wedge i \leq 5) \Rightarrow (6 - i) \geq 0$	UNSAT [OK]

9.8. Nota sobre invariantes

Un invariante debe ser *suficientemente fuerte* para que las tres VCs sean válidas. Un invariante débil como $s \geq 0$ no captura que $i \geq 1$, lo que hace que Z3 encuentre contraejemplos con i negativo para la VC de preservación. La fortaleza del invariante es responsabilidad del programador; el verificador comprueba que el invariante dado es *correcto* (no que sea *completo*).

Capítulo 10

Suite de pruebas

10.1. Organización

Las pruebas se organizan en tres grupos principales:

Cuadro 10.1: Grupos de pruebas

Directorio	Propósito	Tests
tests/backend/	Compilación y ejecución completa	5
tests/semantic/	Errores semánticos esperados	3
tests/verification/	Verificación formal con Z3	3
Total		11

El script `tests/run_tests.sh` ejecuta todos los grupos y reporta el resultado de cada prueba como `[PASS]` o `[FAIL]`.

10.2. Pruebas de backend

Cada prueba compila el fichero `.java`, ejecuta el binario resultante y compara la salida con el valor esperado.

Cuadro 10.2: Pruebas de backend (compilación y ejecución)

Fichero	Qué se verifica	Estado
t01_print.java	<code>System.out.println</code> de enteros	PASS
t02_arithmetic.java	Las cuatro operaciones, módulo y precedencia	PASS
t03_if_else.java	Condicionales con ramas vacías y anidadas	PASS
t04_while.java	Bucles, suma acumulativa, retorno	PASS
t05_functions.java	Llamadas entre métodos, paso de valores	PASS

10.3. Pruebas semánticas

Estos ficheros son programas *incorrectos*: el compilador debe detectar el error semántico y terminar con código de salida distinto de cero.

Cuadro 10.3: Pruebas de errores semánticos

Fichero	Error esperado	Estado
sem01_undeclared.java	Variable usada sin declarar	PASS
sem02_type_error.java	Asignación de double a int sin conversión	PASS
sem03_redecl.java	Redeclaración de variable en el mismo ámbito	PASS

10.4. Pruebas de verificación formal

Cuadro 10.4: Pruebas de verificación con Z3

Fichero	Qué se verifica	VCs	Estado
demo_verified.java	absValue, wrongMethod, sumLoop	7	PASS
ver01_simple.java	Método sin bucles, @post básico	2	PASS
ver02_invariant.java	Invariante de bucle con variante	5	PASS

10.4.1. Salida de ejemplo: demo_verified.java

```
$ ./compiler --verify tests/verification/demo_verified.java
[OK] [absValue] correccion (unsat)
[OK] [sumLoop] iniciacion del invariante (unsat)
[OK] [sumLoop] preservacion del invariante (unsat)
[OK] [sumLoop] uso del invariante (unsat)
[OK] [sumLoop] correccion (unsat)
[OK] [sumLoop] terminacion (unsat)
[BUG] [wrongMethod] correccion (sat) <-- bug detectado
```

10.5. Script de ejecución

El script `tests/run_tests.sh` automatiza todo el proceso:

1. Compila el proyecto con `make`.
2. Ejecuta cada prueba de backend: compila el `.java` con `./compiler`, ejecuta el binario y compara con `expected/`.
3. Ejecuta las pruebas semánticas comprobando que el compilador termina con código $\neq 0$.

4. Ejecuta las pruebas de verificación comprobando que Z3 esté disponible en el PATH.
5. Imprime un resumen: `N/M tests passed`.

```
$ make && tests/run_tests.sh
[PASS] t01_print
[PASS] t02_arithmetic
[PASS] t03_if_else
[PASS] t04_while
[PASS] t05_functions
[PASS] sem01_undeclared
[PASS] sem02_type_error
[PASS] sem03_redecl
[PASS] ver01_simple
[PASS] ver02_invariant
[PASS] demo_verified
11/11 tests passed
```


Apéndice A

Gramática BNF del subconjunto Java

Las reglas se presentan en notación EBNF. Los nombres entre <> son no terminales; los literales entre comillas simples son terminales. Los corchetes denotan opcionalidad; las llaves, cero o más repeticiones; las alternativas se separan con |.

A.1. Estructura del programa

```
1 <Program> ::= 'public' 'class' ID '{' <Members>* '}'
2
3 <Members> ::= <Annotation>
4             | <Method>
5
6 <Annotation> ::= '@pre' <Expr>
7               | '@post' <Expr>
8               | '@invariant' <Expr>
9               | '@variant' <Expr>
```

Listing A.1: Producción raíz

A.2. Declaración de métodos

```
1 <Method> ::= 'public' 'static' <Type> ID '(' [<Params>] ')' <Block>
2           | 'public' 'static' 'void' ID '(' [<Params>] ')' <Block>
3
4 <Params> ::= <Type> ID { ',' <Type> ID }
5
6 <Type> ::= 'int' | 'double' | 'boolean' | 'String[]'
```

Listing A.2: Métodos y parámetros

A.3. Sentencias

```
1 <Block> ::= '{' <Stmt>* '}'
2
```

```

3 <Stmt>      ::= <VarDecl>
4             | <Assign>
5             | <If>
6             | <While>
7             | <Return>
8             | <MethodCall> ';'
9             | <PrintStmt>
10
11 <VarDecl>   ::= <Type> ID ';'
12 <Assign>    ::= ID '=' <Expr> ';'
13
14 <If>        ::= 'if' '(' <Expr> ')' <Block>
15             | 'if' '(' <Expr> ')' <Block> 'else' <Block>
16
17 <While>     ::= 'while' '(' <Expr> ')' <Annotation>* <Block>
18
19 <Return>    ::= 'return' <Expr> ';'
20
21 <PrintStmt> ::= 'System.out.println' '(' <Expr> ')' ';'
22             | 'System.out.print'   '(' <Expr> ')' ';'

```

Listing A.3: Sentencias del subconjunto

A.4. Expresiones

```

1 <Expr>      ::= <Expr> '||' <Expr>
2             | <Expr> '&&' <Expr>
3             | '!' <Expr>
4             | <Expr> ('==' | '!=' | '<' | '<=' | '>' | '>=') <Expr>
5             | <Expr> ('+' | '-') <Expr>
6             | <Expr> ('*' | '/' | '%') <Expr>
7             | '-' <Expr> -- negacion unaria
8             | '(' <Expr> ')'
9             | ID
10            | INT_LIT
11            | REAL_LIT
12            | 'true' | 'false'
13            | 'result' -- solo en anotaciones @post
14            | <MethodCall>
15
16 <MethodCall> ::= ID '(' [<Args>] ')'
17 <Args>       ::= <Expr> { ',' <Expr> }

```

Listing A.4: Gramática de expresiones (precedencia ascendente)

A.5. Anotaciones de verificación

Las anotaciones de verificación (Milestone 5) son un mecanismo de extensión léxica: los tokens `@pre`, `@post`, `@invariant` y `@variant` se reconocen por el analizador léxico (Flex) y se integran en la gramática como sentencias de primer nivel dentro de `Members`, o como prefijos de un `While`.

Las expresiones de anotación comparten la misma gramática que `<Expr>`, con el añadido del símbolo `result` que referencia el valor de retorno del método en la

postcondición.

A.6. Precedencia y asociatividad de operadores

Cuadro A.1: Precedencia (mayor nivel = mayor prioridad)

Nivel	Operadores	Asociatividad
1		izquierda
2	&&	izquierda
3	!	derecha (prefijo)
4	== !=	izquierda
5	< <= > >=	izquierda
6	+ -	izquierda
7	* / %	izquierda
8	unario -	derecha (prefijo)
9	llamada a función, literal, identificador	—

Apéndice B

Referencia de instrucciones x86-64

Se documentan las instrucciones x86-64 generadas por el backend del compilador. Se usa sintaxis AT&T (fuente a la izquierda, destino a la derecha).

B.1. Instrucciones de transferencia de datos

Cuadro B.1: Instrucciones de movimiento

Instrucción	Efecto
<code>movq src, dst</code>	$dst \leftarrow src$ (64 bits)
<code>movl src, dst</code>	$dst \leftarrow src$ (32 bits, zero-extends a 64)
<code>movzbq src, dst</code>	Extiende byte a 64 bits con ceros (<i>zero-extend</i>)
<code>leaq src, dst</code>	$dst \leftarrow$ dirección efectiva de <i>src</i>
<code>pushq src</code>	Empuja <i>src</i> en la pila (<code>%rsp -= 8</code>)
<code>popq dst</code>	Extrae de la pila en <i>dst</i> (<code>%rsp += 8</code>)

B.2. Instrucciones aritméticas enteras

Cuadro B.2: Aritmética entera (64 bits)

Instrucción	Efecto
<code>addq src, dst</code>	$dst \leftarrow dst + src$
<code>subq src, dst</code>	$dst \leftarrow dst - src$
<code>imulq src</code>	$\%rdx : \%rax \leftarrow \%rax \times src$ (con signo)
<code>idivq src</code>	$\%rax \leftarrow \%rdx : \%rax \div src$; $\%rdx \leftarrow$ resto
<code>cqto</code>	Signo-extiende $\%rax$ a $\%rdx : \%rax$ (necesario antes de <code>idivq</code>)
<code>negq dst</code>	$dst \leftarrow -dst$
<code>incq dst</code>	$dst \leftarrow dst + 1$
<code>decq dst</code>	$dst \leftarrow dst - 1$

B.3. Instrucciones de comparación y salto

Cuadro B.3: Comparación y saltos

Instrucción	Efecto / Condición
<code>cmpq src, dst</code>	Calcula $dst - src$; pone flags (SF, ZF, OF)
<code>testq src, dst</code>	Calcula $dst \text{ AND } src$; pone ZF
<code>je label</code>	Salta si $ZF = 1$ (equal / zero)
<code>jne label</code>	Salta si $ZF = 0$
<code>jle label</code>	Salta si $SF \neq OF$ (less, con signo)
<code>jle label</code>	Salta si $ZF = 1$ o $SF \neq OF$
<code>jg label</code>	Salta si $ZF = 0$ y $SF = OF$ (greater)
<code>jge label</code>	Salta si $SF = OF$
<code>jmp label</code>	Salta incondicionalmente
<code>sete al</code>	$al \leftarrow (ZF = 1)$
<code>setne al</code>	$al \leftarrow (ZF = 0)$
<code>setl al</code>	$al \leftarrow (SF \neq OF)$
<code>setle al</code>	$al \leftarrow (ZF \vee SF \neq OF)$
<code>setg al</code>	$al \leftarrow (\neg ZF \wedge SF = OF)$
<code>setge al</code>	$al \leftarrow (SF = OF)$

B.4. Instrucciones de control de flujo y llamadas

Cuadro B.4: Llamadas a función y retorno

Instrucción	Efecto
<code>call label</code>	Empuja <code>%rip</code> +siguiente y salta a <i>label</i>
<code>ret</code>	Extrae dirección de retorno de la pila y salta a ella
<code>nop</code>	Sin operación (relleno de alineación)

B.5. Instrucciones de punto flotante SSE2

Cuadro B.5: Instrucciones SSE2 para double

Instrucción	Efecto
<code>movsd src, dst</code>	Mueve un double
<code>addsd src, dst</code>	$dst \leftarrow dst + src$ (double)
<code>subsd src, dst</code>	$dst \leftarrow dst - src$
<code>mulsd src, dst</code>	$dst \leftarrow dst \times src$
<code>divsd src, dst</code>	$dst \leftarrow dst / src$
<code>ucomisd src, dst</code>	Comparación de doubles (sin signo, pone ZF/PF/CF)
<code>cvttsd2si src, dst</code>	Convierte entero a double
<code>cvttsd2si src, dst</code>	Convierte double a entero (trunca)

B.6. Directivas del ensamblador

Cuadro B.6: Directivas GAS usadas por el compilador

Directiva	Efecto
<code>.globl sym</code>	Exporta el símbolo <i>sym</i> (visible al enlazador)
<code>.text</code>	Sección de código ejecutable
<code>.section .rodata</code>	Sección de datos de solo lectura (strings de formato)
<code>.string "..."</code>	Emite una cadena terminada en <code>\0</code>
<code>.quad n</code>	Emite una constante de 8 bytes
<code>.align n</code>	Alinea la posición actual a 2^n bytes

B.7. Convención de registro de marcos (%rbp)

El compilador usa el marco de pila estándar:

```

Alta dirección
+-----+
| ... args caller |
+-----+ <- %rbp + 16 (primer arg si se pasa en pila)
| return address  |
+-----+ <- %rbp + 8
| saved %rbp      |
+-----+ <- %rbp <-- base del marco de la funcion
| local var 0     |
+-----+ <- %rbp - 8
| local var 1     |
+-----+ <- %rbp - 16
| temporales ...  |

```


+-----+ <- %rsp (top de la pila)

Baja dirección

El tamaño total del marco se calcula como $\lceil n \cdot 8/16 \rceil \times 16$ bytes para mantener `%rsp` alineado a 16 bytes antes de cualquier `call`, según exige la ABI System V AMD64.

Bibliografía

- [1] Gregory J. Chaitin, Marc A. Auslander, Ashok K. Chandra, John Cocke, Martin E. Hopkins, and Peter W. Markstein. Register allocation via coloring. *Computer Languages*, 6(1):47–57, 1981.
- [2] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
- [3] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.